

Object Oriented Programming	start time:
------------------------------------	----------------

Internally, the library class `java.lang.String` stores an array of characters. It also provides a variety of useful methods for comparing, manipulating, and searching text in general.

Learning Objectives (for content & process)

After completing this activity, learners should be able to:

- Identify differences between primitive and object types.
- Identify static vs non-static methods.
- Recognize the source of the "this" parameter in non-static method calls.
- Decide on the necessary fields and methods for a class based on a real object/data.
- Use fields and methods to design a UML diagram.

Before you start, complete the form below to assign a role to each member. If you have 3 people, combine Speaker & Reflector.

Team	Date
Team Roles	Team Member
Recorder: records all answers & questions, and provides copies to team & facilitator.	
Speaker: talks to facilitator and other teams.	
Manager: keeps track of time and makes sure everyone contributes appropriately.	
Reflector: considers how the team could work and learn more effectively.	

Reminders:

1. Note the time whenever your team starts a new section or question.
2. Write legibly & neatly so that everyone can read & understand your responses.

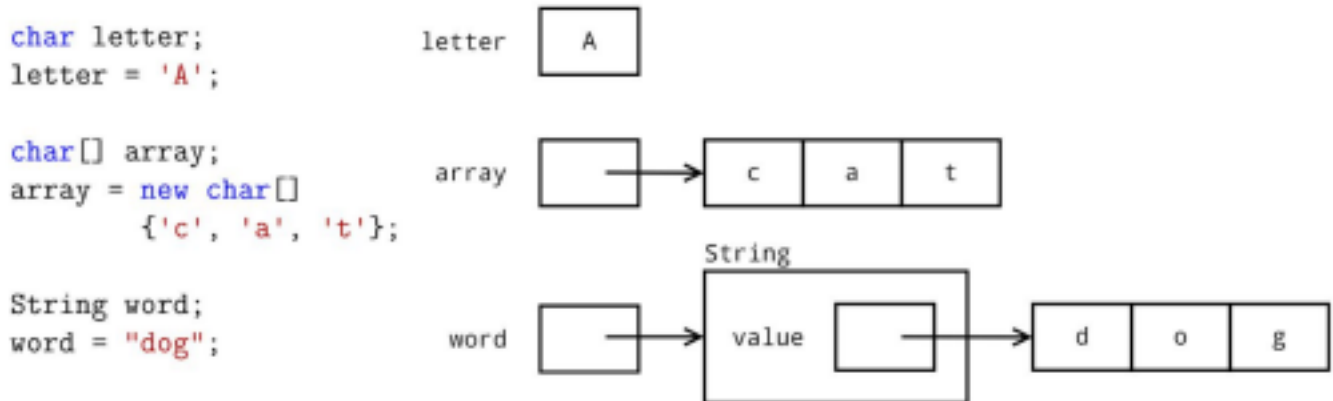
Some diagrams and content from [work](#) by Chris Mayfield and Helen Hu.



© 2018-2020 by Sofia Lemons, sofia.lemons@cs.unh.edu. This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Object Oriented Programming 2020-01-22 pg 2 of 8

(12 min) A. Primitives, Arrays, and Objects	start time:
---	----------------



The primitive type `char` is used to store a single character, which can be a letter, a number, or a symbol. In contrast, the object type `String` **encapsulates** an array of characters.

1. (2 min) How is the syntax (code) different for character literals and `String` literals in Java?

Char is only for single letters, while `String` is used for words.

2. (1 min) List the index for each character in the string above.

d: 0

o: 1

g: 2

3. (1 min) In general, what is the index of the last character in any string?

Subtract 1 from the total amount of characters in the string.

4. (2 min) In the diagram above, String **encapsulates** an array of characters. What do you think it means for an object to **encapsulate** data? Encapsulate means an object stores data inside of itself



© 2018-2020 by Sofia Lemons, sofia.lemons@cs.unh.edu. This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Object Oriented Programming 2020-01-22 pg 3 of 8

5. (2 min) How do you access the data within a String's encapsulated character array? By

using `charAt()`

```
String str;  
str = "Hi";  
str += "!";
```

```
char let;  
let = 'X';
```

```
int num;  
num = -1;
```

```
int foo;  
foo = num;
```

```
String hmm;  
hmm = "Hi!";
```

6. (2 min) Recall that the `==` operator compares the value of two variables or literals. What will the result of each of the following comparisons be?

```
let == '!'; false
```

```
let == 'X'; true
```

```
num == foo; true
```

```
str == hmm; false
```

```
str == str; true
```

7. (2 min) Object type variables like `str` store a reference (or memory address) for a particular object, rather than storing the object itself. Use this information to explain why `str == hmm` gives a different result than `str == str`.

`str == hmm` is false because they reference different objects. `str == str` is true because it references the same object



© 2018-2020 by Sofia Lemons, sofia.lemons@cs.unh.edu. This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Object Oriented Programming 2020-01-22 pg 4 of 8

(14 min) B. Static & Non-Static Methods	start time:
--	----------------

Method		Description
<code>charAt(int)</code>	<code>char</code>	Returns the char value at the specified index of this string.
<code>indexOf(String)</code>	<code>int</code>	Returns the index within this string of the first occurrence of the specified character.
<code>length()</code>	<code>int</code>	Returns the length of this string. The length is equal to the number of Unicode code units in the string.
<code>substring(int, int)</code>	<code>String</code>	Returns a new string that is a substring of this string (from <code>beginIndex</code> to <code>endIndex - 1</code>).
<code>toUpperCase()</code>	<code>String</code>	Converts all of the characters in this String to upper case using the rules of the default

		locale.
--	--	---------

Each method listed above is `non-static`. That is, they have an **implicit parameter** named `this` that is passed automatically. (Note: There are many other `String` methods not listed above.)

1. (3 min) If the variable `str` contains the string "hello world", then what is the return value of the following method calls?

a. `str.charAt(8)` = `r`

d. `str.substring(4, 7)` = `(o, w)`

b. `str.indexOf("wo")` = `6`

e. `str.toUpperCase()` = `"HELLO WORLD"`

c. `str.length()` = `11`

2. (2 min) Explain what precedes the dot operator (`.`) in the expressions above. What does it have to do with the keyword `this` described above? The object name comes before the dot. That object is automatically passed as `this` to the method



© 2018-2020 by Sofia Lemons, sofia.lemons@cs.unh.edu. This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Object Oriented Programming 2020-01-22 pg 5 of 8

To call a `static` method, you write `ClassName.methodName`, for example: `Math.abs(-5)`

To call a non-static method, you write `objectName.methodName`, for example: `str.charAt(8)`

Methods can be designed either way. Most `String` methods are non-static, because that makes the code easier to read. From the `charAt` method's point of view, `str` is the `this` object.

3. (3 min) Indicate whether the following methods are static or non-static.

Method	Static or non-static
<code>scan.nextInt()</code>	Non-static
<code>str.equals("something")</code>	Non-static
<code>Integer.parseInt("25")</code>	Static
<code>Math.sqrt(2)</code>	Static
<code>"text".toUpperCase()</code>	Non-static

4. (3 min) How many total parameters does each method call below have? (Hint: none of them have zero.)

a. `scan.nextInt() = 1`

d. `str.substring(4, 7) = 3`

b. `Math.sqrt(2) = 1`

e. `str.equals("something") = 2`

c. `Math.pow(10, 3) = 2`

5. (3 min) The parameters listed in a method signature can be any type of data the programmer

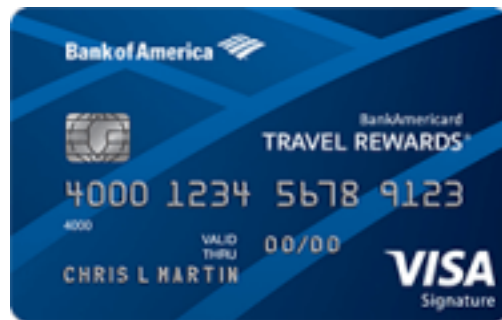
chooses. What is the restriction on the type of the implicit `this` parameter for non-static methods? The `this`-parameter has to be the same type as the class the method belongs to



© 2018-2020 by Sofia Lemons, sofia.lemons@cs.unh.edu. This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Object Oriented Programming 2020-01-22 pg 6 of 8

(16 min) C. Designing Objects & Classes	start time:
---	----------------



Classes are the specification for what data and methods objects of that type can have. For example:

`String` is the class that defines generally what any string can store or do, and `str` references a specific `String` object like one containing the data "hello world".

In this section, you will design a new class (`CreditCard`) to represent credit cards. Each `CreditCard` object would represent an individual's specific credit card.

1. (2 min) List any information that `CreditCard` objects would need to store (encapsulate.) Give the data type for each. These are called **fields** or **instance variables**.

2. (2 min) Which of the fields would need to be defined immediately when a `CreditCard` object is created in order for it to validly represent a credit card? Which would not be needed right away?

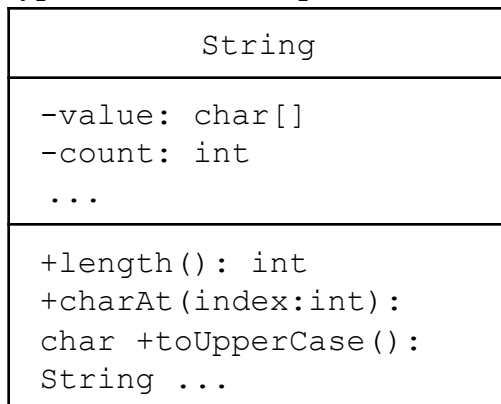
3. (2 min) Which of the fields would need to be protected from having invalid values? What would the acceptable ranges of values for those fields be?



© 2018-2020 by Sofia Lemons, sofia.lemons@cs.unh.edu. This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Object Oriented Programming 2020-01-22 pg 7 of 8

One way to represent the design of a class is **UML** (Unified Modeling Language) diagrams. These illustrate the **fields** and **methods** that belong to a class (and therefore all objects of that type.) Below is a simplified UML diagram for the Java `String` class.



4. (3 min) The `char` array inside a `String` object can't be accessed or modified directly (programmers have to use the class' methods to work with it.) What are some of the problems that might occur if programmers could modify the `char` array directly?

5. (2 min) List at least two methods that would be appropriate for the `CreditCard` class. Include arguments and return types.

6. (5 min) Design a UML diagram for your `CreditCard` class, including the fields and methods you specified above. Make sure to use the `name: type` format for methods, fields, and parameters.



© 2018-2020 by Sofia Lemons, sofia.lemons@cs.unh.edu. This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Object Oriented Programming 2020-01-22 pg 8 of 8

Reflector: Get an '[Evidence of Competencies](#)' form and provide one piece of positive evidence for each member of the group. Then, complete the section for the 'Assessment of Group.' Share the document with each of your group members. All students should be collecting pieces of evidence for their mid-semester and end of semester reflections.



© 2018-2020 by Sofia Lemons, sofia.lemons@cs.unh.edu. This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.